

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 1 Practical No: 4

Roll no: 38 PRN No: 12413458 Name of Student: **Vedanti Raut**

Subject Name : Deep Learning

Problem Statement : Develop an LSTM-based model for time-series forecasting using stock price, weather, or sales datasets.

Algorithm :

1. Load the Dataset – Import the sales/stock/weather dataset and select the relevant time-series column.
2. Preprocess the Data – Convert the date column into a proper time format and handle missing values.
3. Normalize the Data – Scale the numerical values to a suitable range, such as 0 to 1.
4. Create Sequences – Divide the time-series data into input sequences and corresponding target values using a fixed time window.
5. Split the Dataset – Divide the sequences into training and testing sets while maintaining the time order.
6. Reshape the Data – Convert the input data into the 3D format required by LSTM: samples × time steps × features.
7. Build the LSTM Model – Create an LSTM layer followed by a Dense output layer for predicting the next value.
8. Compile the Model – Use an optimizer such as Adam and Mean Squared Error (MSE) as the loss function.
9. Train the Model – Train the LSTM using the training sequences for a specified number of epochs.
10. Make Predictions – Use the trained model to predict values for the test sequences.
11. Inverse Transform – Convert the normalized predictions back to their original scale.
12. Evaluate Performance – Calculate metrics such as MSE, RMSE, and MAE.
13. Visualize Results – Plot the actual values against the predicted values to analyze forecasting performance.

Code :

```
#1 Import Libraries and Download Dataset
import kagglehub
from kagglehub import KaggleDatasetAdapter

import tensorflow as tf
import numpy as np
import os
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error

path = kagglehub.dataset_download("rohitsahoo/sales-forecasting")

print("Dataset path:", path)
```

Show hidden output

```
#2 Load and Inspect Dataset
print('Contents of the dataset directory:')
for item in os.listdir(path):
    print(item)
```

Show hidden output

```
data_file_path = os.path.join(path, "train.csv")
df = pd.read_csv(data_file_path)
print("First 5 records:")
print(df.head())
```

```
#3 Convert Date to Date-Time Format
df['Order Date'] = pd.to_datetime(df['Order Date'], dayfirst=True)

print(df[['Order Date', 'Sales']].head())
print(df['Order Date'].dtype)
```

Show hidden output

```
#4 Aggregate Sales by Month
monthly_sales = (df.groupby(pd.Grouper(key='Order Date', freq='ME'))['Sales'].sum().reset_index())

print(monthly_sales.head(10))
```

Show hidden output

```
#3 Visualize Monthly Sales Trend
plt.figure(figsize=(12, 5))

plt.plot(
    monthly_sales['Order Date'],
    monthly_sales['Sales'],
    marker='o'
)

plt.title('Monthly Sales Trend')
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.xticks(rotation=45)
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
#6 Split Data into Training and Testing Sets
train_size = int(len(sales) * 0.8)

train_data = sales[:train_size]
test_data = sales[train_size:]

print("Training data:", len(train_data))
print("Testing data:", len(test_data))
```

Show hidden output

```
#7 Normalize Sales Data
scaler = MinMaxScaler()

train_scaled = scaler.fit_transform(
    train_data.reshape(-1, 1)
)

test_scaled = scaler.transform(
    test_data.reshape(-1, 1)
)

print("Original first value:", train_data[0])
print("Scaled first value:", train_scaled[0])
```

```
#8 Create LSTM Time-Series Sequences
def create_sequences(data, time_steps=12):
    X = []
    y = []

    for i in range(len(data) - time_steps):
        X.append(data[i:i + time_steps])
        y.append(data[i + time_steps])

    return np.array(X), np.array(y)

X_train, y_train = create_sequences(
    train_scaled,
    time_steps=12
)

test_input = np.concatenate(
    [train_scaled[-12:], test_scaled]
)

X_test, y_test = create_sequences(
    test_input,
    time_steps=12
)

print("X_train shape:", X_train.shape)
print("y_train shape:", y_train.shape)

print("X_test shape:", X_test.shape)
print("y_test shape:", y_test.shape)
```

```
#9 Build LSTM Model
model = tf.keras.Sequential([tf.keras.layers.Input(shape=(12, 1)),tf.keras.layers.LSTM(64),
tf.keras.layers.Dense(32, activation='relu'),tf.keras.layers.Dense(1)])

model.summary()
```

Show hidden output

```
#10 Train the LSTM Model
model.compile(optimizer='adam',loss='mean_squared_error')

history = model.fit(X_train,y_train,epochs=50,batch_size=4,validation_split=0.2)
```

Show hidden output

```
#11 Visualize Training Loss
plt.figure(figsize=(10, 5))

plt.plot(history.history['loss'])

plt.title('LSTM Training Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')

plt.grid(True)
plt.show()
```

```
#12 Generate Predictions
predictions = model.predict(X_test)

predictions = scaler.inverse_transform(predictions)
actual = scaler.inverse_transform(y_test)
```

Show hidden output

```
#13 Evaluate Model Using RMSE
rmse = np.sqrt(mean_squared_error(actual, predictions))

print("RMSE:", rmse)
```

Show hidden output

```
#14 Compare Actual and Predicted Sales
plt.figure(figsize=(12, 5))

plt.plot(actual, label='Actual Sales', marker='o')
plt.plot(predictions, label='Predicted Sales', marker='o')

plt.title('Actual vs Predicted Sales')
plt.xlabel('Test Month')
plt.ylabel('Sales')

plt.legend()
plt.grid(True)
plt.show()
```

Output :

#2 Load and Inspect Dataset

First 5 records:

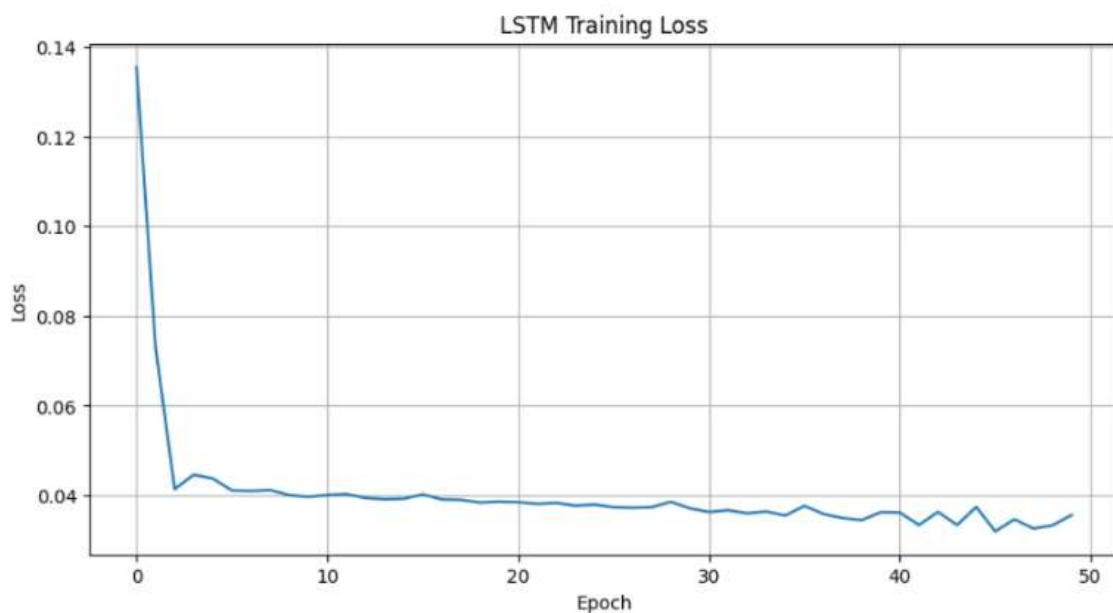
Row	ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	\
0	1	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	
1	2	CA-2017-152156	08/11/2017	11/11/2017	Second Class	CG-12520	
2	3	CA-2017-138688	12/06/2017	16/06/2017	Second Class	DV-13045	
3	4	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	
4	5	US-2016-108966	11/10/2016	18/10/2016	Standard Class	SO-20335	

	Customer Name	Segment	Country	City	State	\
0	Claire Gute	Consumer	United States	Henderson	Kentucky	
1	Claire Gute	Consumer	United States	Henderson	Kentucky	
2	Darrin Van Huff	Corporate	United States	Los Angeles	California	
3	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	
4	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	

	Postal Code	Region	Product ID	Category	Sub-Category	\
0	42420.0	South	FUR-BO-10001798	Furniture	Bookcases	
1	42420.0	South	FUR-CH-10000454	Furniture	Chairs	
2	90036.0	West	OFF-LA-10000240	Office Supplies	Labels	
3	33311.0	South	FUR-TA-10000577	Furniture	Tables	
4	33311.0	South	OFF-ST-10000760	Office Supplies	Storage	

	Product Name	Sales
0	Bush Somerset Collection Bookcase	261.9600
1	Hon Deluxe Fabric Upholstered Stacking Chairs,...	731.9400
2	Self-Adhesive Address Labels for Typewriters b...	14.6200
3	Bretford CR4500 Series Slim Rectangular Table	957.5775
4	Eldon Fold 'N Roll Cart System	22.3680

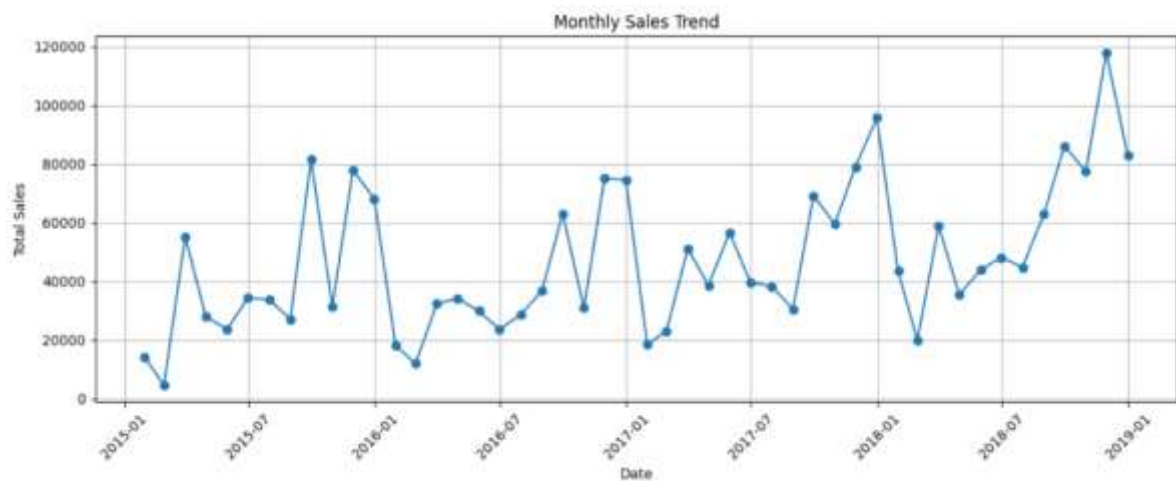
#3 Convert Date to Date-Time Format



#4 Aggregate Sales by Month

	Order Date	Sales
0	2015-01-31	14205.7070
1	2015-02-28	4519.8920
2	2015-03-31	55205.7970
3	2015-04-30	27906.8550
4	2015-05-31	23644.3030
5	2015-06-30	34322.9356
6	2015-07-31	33781.5430
7	2015-08-31	27117.5365
8	2015-09-30	81623.5268
9	2015-10-31	31453.3930

#5 Visualize Monthly Sales Trend



#6 Split Data into Training and Testing Sets

```
Training data: 38
Testing data: 10
```

#7 Normalize Sales Data

```
Original first value: 14205.707
Scaled first value: [0.10618172]
```

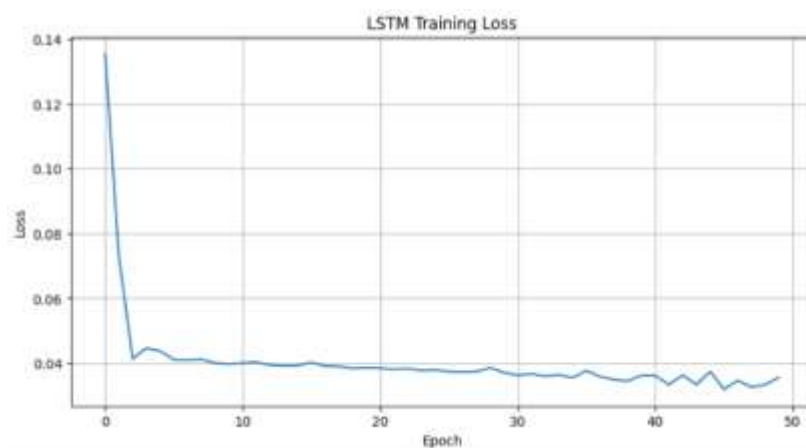
#9 Build LSTM Model

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	16,896
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 19,009 (74.25 KB)
Trainable params: 19,009 (74.25 KB)
Non-trainable params: 0 (0.00 B)

#11 Visualize Training Loss



#14 Compare Actual and Predicted Sales

